

Processador SAP-1

Relatório de Verificação Funcional por Simulação

Implementação em Verilog · Simulação em ModelSim ASE 2020.1

Plataforma-alvo: Terasic DE10-Lite (Intel MAX 10)

Disciplina: _____

Integrantes: _____

Data: ____ / ____ / ____

1. Introdução

Este relatório documenta a verificação funcional do processador didático SAP-1 (Simple-As-Possible), implementado em Verilog e validado por simulação antes da gravação na FPGA. O foco não é a arquitetura em si, mas a comprovação, por formas de onda e testes auto-verificáveis, de que cada componente e o sistema completo se comportam conforme o especificado.

Características relevantes do processador para a verificação:

- **Barramento único de 8 bits** (barramento W), implementado por multiplexador.
- **Memória RAM 16×8** (16 posições de 8 bits), somente leitura em execução, compartilhada entre programa e dados (arquitetura de Von Neumann).
- **Cinco instruções:** LDA (0000), ADD (0001), SUB (0010), OUT (1110) e HLT (1111); cada palavra é [opcode(4) | operando(4)], com endereçamento direto.
- **Unidade de controle** baseada em máquina de estados (contador de anel) com seis estados de temporização T1–T6: T1–T3 realizam a busca (iguais para toda instrução) e T4–T6 a execução (dependente do opcode).
- **Palavra de controle de 12 bits** (CON), gerada a cada estado, que comanda quem escreve e quem lê no barramento.

2. Metodologia de simulação

2.1. Ferramentas

A verificação foi conduzida principalmente no ModelSim ASE 2020.1 (Intel FPGA Starter Edition), o simulador integrado ao Quartus Prime Lite. As formas de onda apresentadas neste relatório foram capturadas diretamente da janela Wave do ModelSim. Como referência cruzada, os mesmos testbenches também foram executados em Icarus Verilog, obtendo-se resultados idênticos.

2.2. Testbenches auto-verificáveis

Foram escritos 14 testbenches — um para cada componente e um para o sistema integrado. Todos são auto-verificáveis: comparam o valor obtido com o esperado e emitem PASSOU ou FALHOU no transcript, encerrando com \$stop. Não geram arquivos de dump (VCD), seguindo o fluxo nativo do ModelSim, no qual os sinais são observados diretamente na base de dados de simulação (WLF).

2.3. Modos de simulação empregados

A verificação usou quatro modos complementares:

- **Simulação unitária (por componente):** cada bloco é compilado e exercitado isoladamente pelo seu testbench, permitindo isolar falhas. Executada por componente com o script parametrizado run_one.do.
- **Simulação integrada (sistema completo):** o testbench tb_sap1 instancia o processador inteiro, carrega um programa na RAM e verifica a saída final (instrução OUT) e a parada correta em HLT.
- **Formas de onda didáticas:** scripts wave_*.do configuram a janela Wave com radix personalizados — o estado do anel aparece como T1...T6 e o opcode como mnemônico (LDA, ADD, ...) — e os registradores em decimal, facilitando a leitura.
- **Execução em lote:** o script run_all_tb.do compila e roda os 14 testbenches em sequência, consolidando os resultados PASSOU/FALHOU.

Radix personalizados definidos nos scripts de onda (trecho):

```
radix define ESTADO {  
    6'b000001 "T1", 6'b000010 "T2", 6'b000100 "T3",  
    6'b001000 "T4", 6'b010000 "T5", 6'b100000 "T6" }  
radix define OPCODE {  
    4'b0000 "LDA", 4'b0001 "ADD", 4'b0010 "SUB",  
    4'b1110 "OUT", 4'b1111 "HLT" }
```

Comandos típicos de reprodução (dentro da pasta tb_model):

```
# simulacao integrada com ondas do sistema:  
vsim -do wave_sap1.do  
# ondas de um componente (ex.: contador de programa):  
vsim -do wave_pc.do  
# lote com todos os testbenches:  
vsim -c -do run_all_tb.do
```

2.4. Testbenches implementados

Testbench	Alvo	Tipo de verificação
tb_program_counter	Program Counter	contagem, reset, pausa
tb_mar	MAR	carga/retenção de endereço
tb_ram_16x8	RAM 16×8	leitura do conteúdo por endereço
tb_instruction_register	Registrador de instrução	separação opcode operando
tb_accumulator	Acumulador (A)	carga/retenção
tb_register_b	Registrador B	carga/retenção
tb_adder_subtractor	ULA	soma, subtração,

		wraparound 8 bits
tb_output_register	Registrador de saída	carga/retenção
tb_controller_sequence r	Controlador/Sequenciador	estados e palavra de controle
tb_seg7 / tb_seg7_instr	Decodificadores 7-seg	mapeamento de dígitos/letras
tb_clock_divider / tb_debouncer	Apoio de FPGA	divisão de clock / antirruído
tb_sap1	Sistema completo	execução de programa fim-a-fim

3. Verificação dos componentes

Esta seção apresenta as formas de onda da simulação unitária dos principais componentes. Um padrão de projeto recorrente é o registrador com carga ativa em baixo: o bloco captura o barramento apenas quando seu sinal de carga vale 0 (na borda de subida do clock) e mantém o valor caso contrário; o reset assíncrono zera a saída. Esse padrão é compartilhado por MAR, IR, acumulador, registrador B e registrador de saída.

3.1. Program Counter (PC)

O que se espera: um contador de 4 bits que (i) incrementa quando habilitado, (ii) zera no reset e (iii) congela quando desabilitado. Por ser de 4 bits, deve contar de 0 a 15 e dar a volta (aritmética módulo 16), o que casa com as 16 posições da memória — o PC nunca aponta para um endereço inexistente.

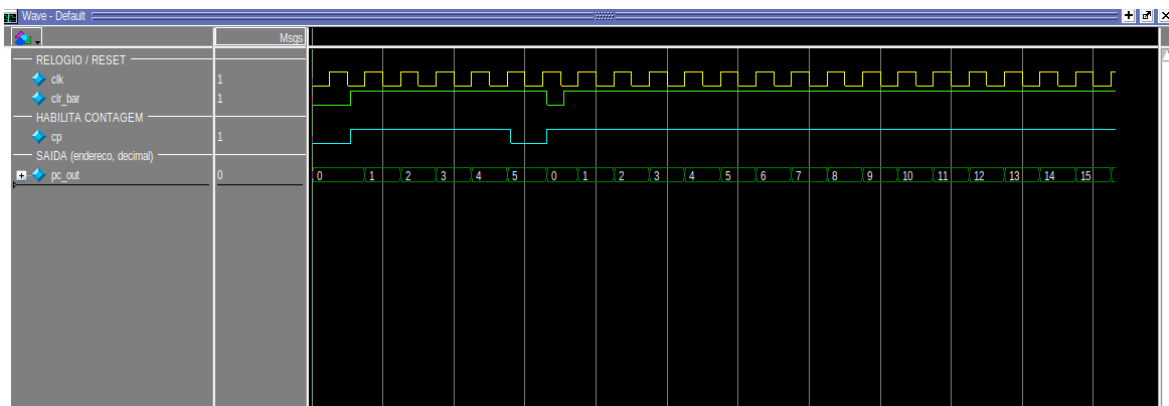


Figura 1 — Contador de programa: contagem, reset e pausa.

A onda comprova as três operações. Com o sinal de habilitação $cp = 1$, a saída avança 0, 1, 2, 3, ... a cada borda de subida do clock — exatamente um

incremento por ciclo, sem saltos. Quando `clr_bar` é levado a 0, o contador retorna a 0 imediatamente, sem esperar o clock (reset assíncrono), como esperado de um sinal de inicialização. Com `cp = 0`, a contagem é congelada e o valor permanece estável mesmo com o clock ativo — esse congelamento é fundamental: é o mesmo mecanismo usado para o PC não avançar fora dos momentos certos do ciclo de instrução. Por fim, ao chegar em 15 a saída retorna a 0, confirmando o wraparound de 4 bits.

3.2. Registrador de Endereço da Memória (MAR)

O que se espera: um registrador de 4 bits que capture o endereço presente no barramento apenas quando sua carga estiver ativa (`lm_bar = 0`) e o segure estável durante toda a leitura da RAM. O ponto crítico a verificar é a retenção: o MAR precisa manter o endereço mesmo depois que o barramento muda, senão a RAM leria a posição errada.

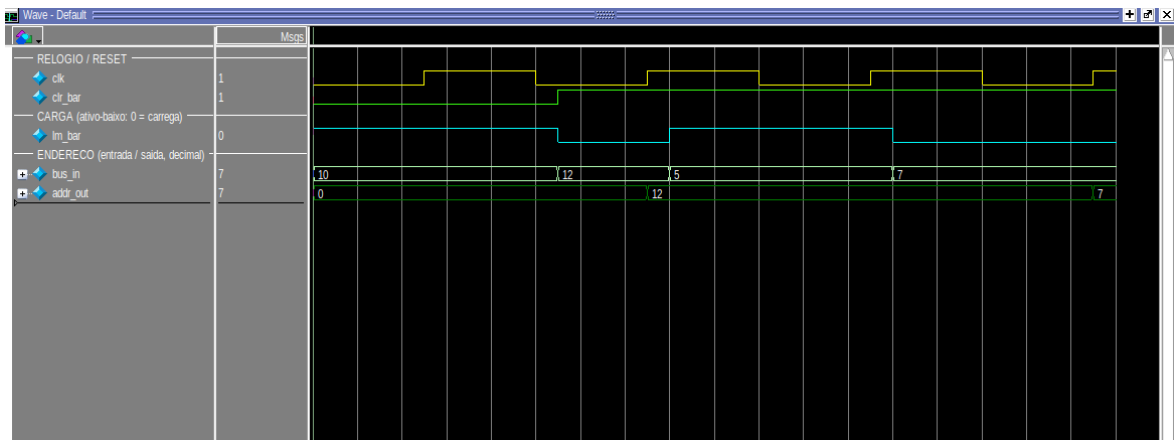


Figura 2 — MAR: carga de endereço e retenção.

A onda confirma o comportamento esperado. Com `lm_bar = 0` o MAR carrega o valor da entrada (por exemplo, 12) na subida do clock. Em seguida, embora a entrada mude para 5, a saída permanece em 12, pois `lm_bar` voltou a 1 — a retenção funciona. Só há nova atualização quando a carga é reativada (passando a 7). Esse endereço estável é justamente o que a RAM usa: durante uma instrução, o MAR é carregado duas vezes — no T1 com o endereço da instrução (vindo do PC) e no T4 com o operando (vindo do IR).

3.3. Memória RAM 16×8

O que se espera: uma memória de 16 palavras de 8 bits que devolva, para cada endereço, exatamente o byte gravado — sem clock, de forma combinacional. Espera-se ainda reconhecer, na palavra lida, os dois campos [opcode | operando] e confirmar que o operando é um endereço (endereçamento direto), não um valor imediato.

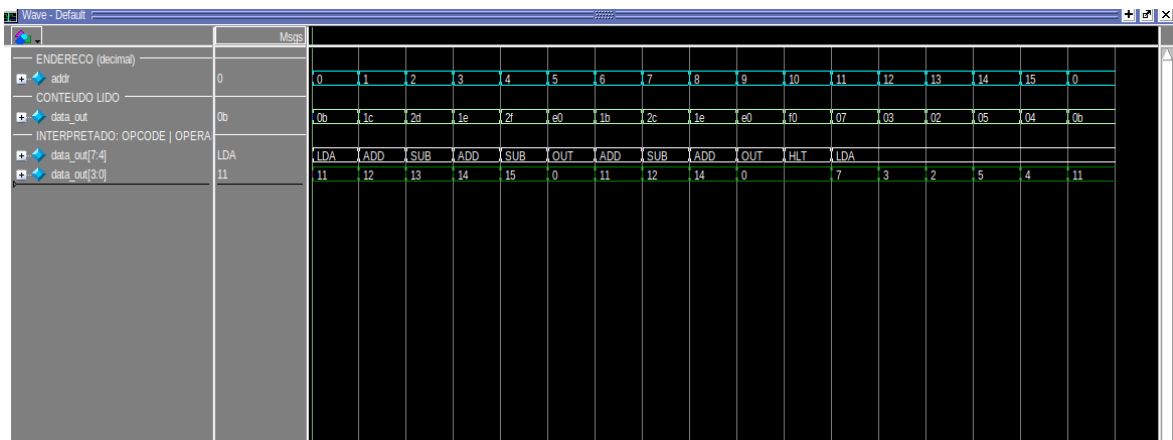


Figura 3 — RAM: leitura sequencial de todos os endereços (Programa 2).

O testbench percorre os 16 endereços e observa o conteúdo. A onda funciona como um "dump" do programa: endereço 0 → LDA 11, 1 → ADD 12, 2 → SUB 13, e assim por diante até HLT no endereço 10; os endereços 11 a 15 guardam os dados (7, 3, 2, 5, 4). Isso comprova, de forma visual, dois pontos conceituais importantes. Primeiro, o endereçamento direto: a linha interpretada mostra que o operando de "LDA 11" é o endereço 11 — para saber o valor, é preciso ir ler a posição 11 (que contém 7). Segundo, a coexistência de programa e dados na mesma memória (Von Neumann), o que explica por que os 16 slots precisam ser compartilhados. Como o SAP-1 básico não tem instrução de escrita (STA), essa memória é apenas de leitura em execução — na prática, uma ROM inicializada por um bloco inicial.

3.4. Registrador de Instrução (IR)

O que se espera: que a palavra de 8 bits vinda da memória seja capturada e dividida em dois campos de 4 bits — os bits altos formam o opcode (o que fazer) e os baixos, o operando (onde). É a etapa de "decodificação" do SAP-1.

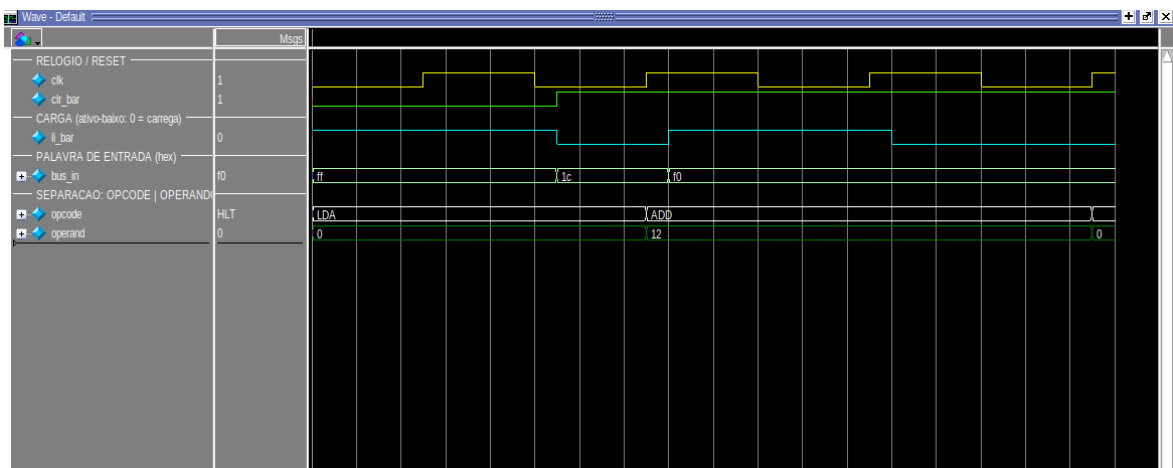


Figura 4 — IR: carga da palavra e separação opcode | operando.

A onda confirma a separação esperada. Ao carregar 0x1C (0001_1100) a saída se decompõe em opcode = ADD (0001) e operando = 12 (1100); ao carregar 0xF0 (1111_0000), em opcode = HLT e operando = 0. A partir daí os dois campos seguem caminhos diferentes: o opcode vai para a unidade de controle (que decide a sequência de microoperações) e o operando é colocado no barramento para virar o próximo endereço no MAR. Note que a saída só muda quando $li_bar = 0$, mantendo a instrução estável durante todos os estados de execução — a unidade de controle precisa do opcode fixo de T4 a T6.

3.5. Acumulador (A) e Registrador B

O que se espera: dois registradores de 8 bits idênticos em comportamento. O acumulador guarda o resultado corrente das contas; o B guarda o segundo operando da ULA. Espera-se, novamente, o padrão carrega-quando-habilitado / mantém-caso-contrário — essencial para que o valor de A sobreviva entre uma instrução e outra.

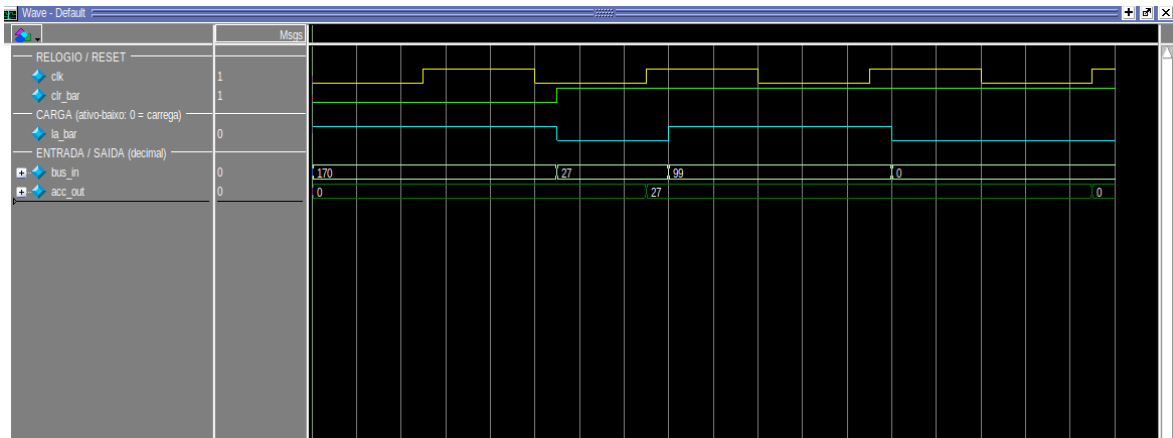


Figura 5 — Acumulador: carrega quando habilitado e mantém quando não.

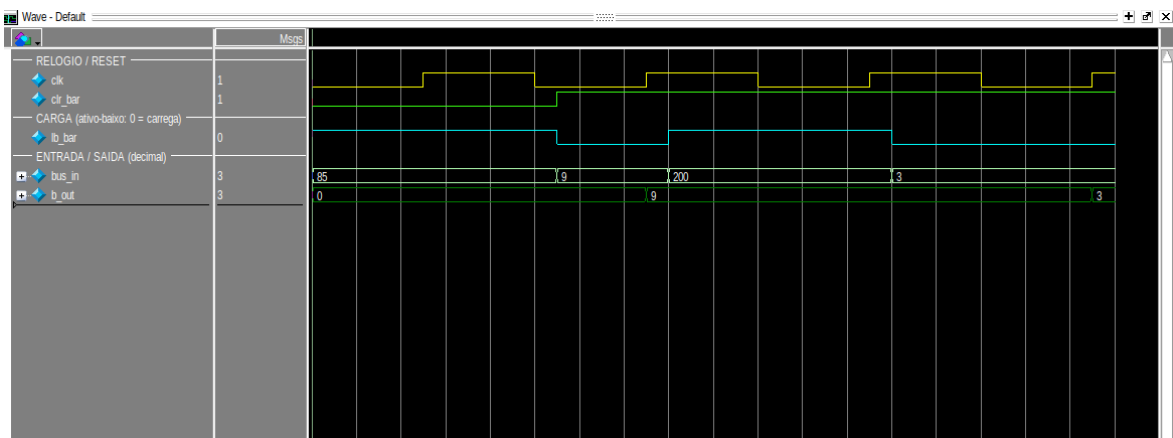


Figura 6 — Registrador B: mesmo padrão de carga/retenção.

As duas ondas confirmam o esperado. No acumulador, com $la_bar = 0$ a saída assume 27 e, mesmo com a entrada mudando para 99, permanece em 27 enquanto a carga não for reativada — depois carrega 0. O registrador B repete o padrão (carrega 9, mantém com a entrada em 200, depois carrega 3). Essa retenção é o que torna possível a aritmética acumulativa: em um ADD, o valor antigo de A é somado ao novo dado e o resultado volta para A. Vale destacar a diferença de papéis — enquanto o B é apenas um "copo" temporário para o segundo operando, o A é o registrador de trabalho, lido e reescrito repetidamente ao longo do programa.

3.6. Unidade Lógico-Aritmética (ULA)

O que se espera: um bloco puramente combinacional que calcule $A + B$ (quando $su = 0$) ou $A - B$ (quando $su = 1$), em 8 bits. Como não há bits extras, espera-se wraparound: somas acima de 255 e subtrações abaixo de 0 devem "dar a volta" (aritmética módulo 256 / complemento de dois).

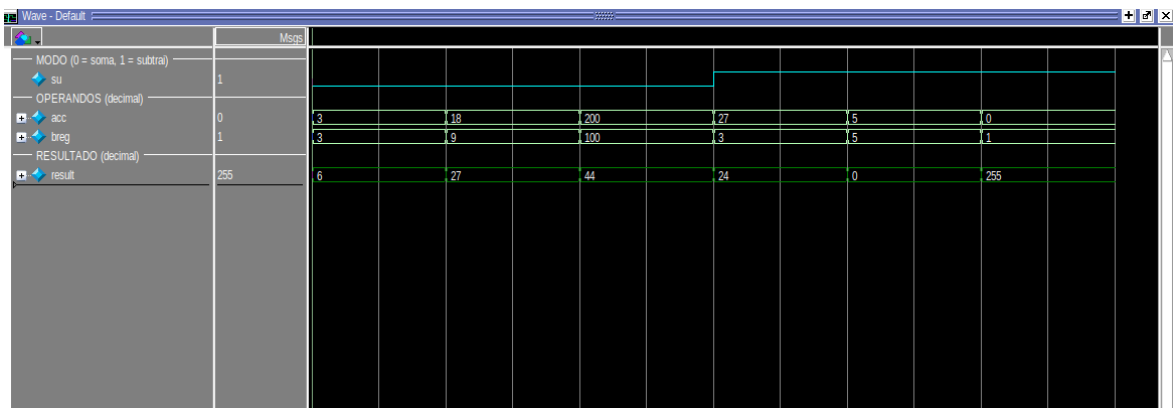


Figura 7 — ULA: soma, subtração e wraparound de 8 bits.

A onda valida todos os casos, inclusive os de borda. Com $su = 0$ (soma): $3+3 = 6$ e $18+9 = 27$, resultados diretos; e $200+100 = 44$, que é exatamente $300 - 256$ — o wraparound esperado. Com $su = 1$ (subtração): $27-3 = 24$, $5-5 = 0$ e, o caso mais instrutivo, $0-1 = 255$, que é a representação de -1 em complemento de dois com 8 bits. Por ser combinacional, o resultado acompanha as entradas sem atraso de clock; ele só chega ao barramento — e, portanto, ao acumulador — quando o sinal Eu é ativado (no estado T6 das instruções ADD/SUB). Esse limite de 8 bits é a razão de o SAP-1 só operar com valores de 0 a 255.

3.7. Controlador / Sequenciador

O que se espera: o bloco mais crítico do processador. Espera-se um contador de anel que percorra $IDLE \rightarrow T1 \rightarrow \dots \rightarrow T6 \rightarrow IDLE$ e, em cada estado, emita uma palavra de controle de 12 bits (CON) com a combinação exata de sinais daquele passo. A palavra deve depender apenas do estado e

do opcode — nunca do dado — garantindo comportamento determinístico. Espera-se também que HLT trave a máquina em T4.

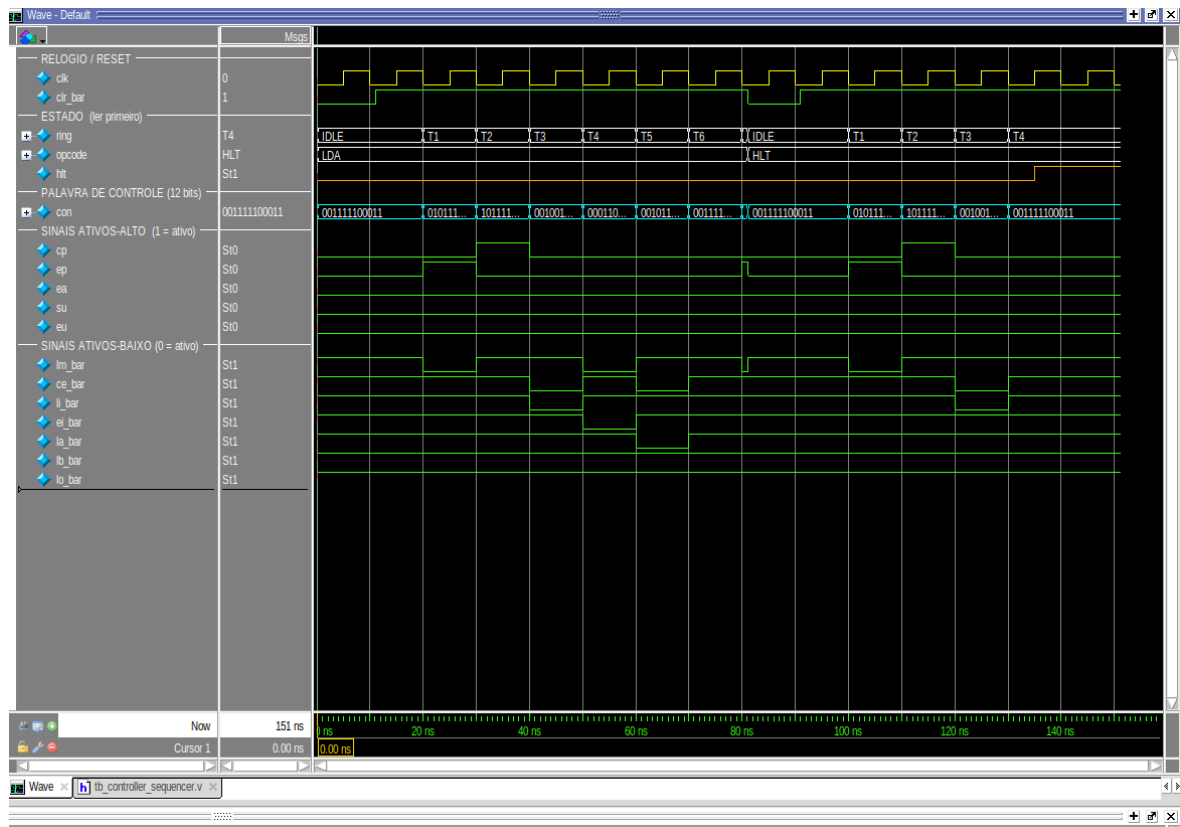


Figura 8 — Máquina de estados e palavra de controle de 12 bits por estado.

A onda confirma cada expectativa. O anel percorre IDLE → T1 → T2 → ... → T6 → IDLE, e a palavra CON assume um valor distinto e bem definido em cada estado. Os sinais ativos em alto (cp, ep, ea, su, eu) aparecem como pulsos nos estados corretos — por exemplo, ep no T1 (o PC fala) e cp no T2 (o PC incrementa) —, enquanto os sinais ativos em baixo (as cargas) descem a 0 apenas quando devem atuar. Um detalhe importante confirmado aqui: a palavra de repouso não é toda-zero, justamente porque as cargas são ativas em baixo. Por fim, ao aplicar HLT a máquina permanece travada em T4, comprovando a parada correta. É desta tabela de palavras que nascem, coordenadamente, todos os movimentos vistos nas ondas anteriores.

4. Verificação do sistema completo

Com os componentes validados, o testbench `tb_sap1` exercita o processador inteiro. Para cada programa, a RAM é inicializada, a simulação roda até o HLT e a saída da instrução OUT é comparada, por assert automático, ao valor esperado. As ondas a seguir usam radix didáticos: o estado aparece como

T1...T6, o opcode como mnemônico e o datapath (PC, MAR, A, B, ULA, saída) em decimal.

4.1. Modelo temporal esperado (como prever a onda)

Antes de analisar cada programa, convém fixar o modelo de tempo esperado — é ele que permite "prever" a onda e depois conferir. Duas regras de temporização governam tudo:

- O **anel de estados avança na borda de descida** do clock (negedge): é quando a máquina passa de T1 para T2, de T2 para T3, e assim por diante.
- Os **registradores carregam na borda de subida** (posedge), quando seu sinal de carga está ativo. Ou seja, o controle "prepara" o sinal em um estado e o dado é efetivamente capturado no flanco seguinte.

Com isso, cada instrução consome exatamente 6 estados (T1–T6). Os três primeiros são a busca, idênticos para toda instrução:

```
T1: PC -> MAR      (endereço da instrução vai para o MAR)
T2: PC = PC + 1     (aponta para a próxima instrução)
T3: RAM -> IR       (a instrução é lida e decodificada)
```

Os três últimos (execução) dependem do opcode:

```
LDA: T4 IR.op -> MAR   T5 RAM -> A      (A muda no T5)
ADD: T4 IR.op -> MAR   T5 RAM -> B      T6 ULA(A+B) -> A
SUB: T4 IR.op -> MAR   T5 RAM -> B      T6 ULA(A-B) -> A
OUT: T4 A -> registrador de saída
HLT: T4 trava o anel  (congela em T4, sem parar o clock)
```

Consequência prática na leitura da onda: em um ADD/SUB, o novo valor do acumulador só aparece no T6 e, portanto, torna-se visível como valor "assentado" já no T1 da instrução seguinte. Por isso, ao ler as ondas, o acumulador parece mudar "um passo depois" — não é atraso indevido, é o modelo esperado. Esse detalhe é a principal fonte de confusão ao interpretar formas de onda de processadores, e a simulação o torna concreto.

Número de ciclos esperado: como cada instrução leva 6 estados, um programa com N instruções (contando o HLT) deve parar em torno de $6 \times N$ ciclos. Usaremos essa conta para conferir cada caso.

4.2. Programa 1 — $3^3 = 27$

Potência por somas sucessivas; exibe 9 (3^2) e depois 27 (3^3).

Listagem (endereço : instrução):

```
0 LDA 12 ; A <- 3      8 ADD 12 ; A <- 27
1 ADD 12 ; A <- 6      9 ADD 15 ; A <- 27 (3^3)
2 ADD 12 ; A <- 9 (3^2) 10 OUT      ; mostra 27
3 OUT      ; mostra 9   11 HLT
4 LDA 13 ; A <- 9      12 dado = 3
5 ADD 13 ; A <- 18     13 dado = 9
```

6	ADD 13	; A <- 27	14	dado = 3
7	SUB 14	; A <- 24	15	dado = 0

Resultado esperado (dedução): $3^3 = 3 \times 3 \times 3 = 27$. Sem multiplicação nativa, o cálculo é feito em duas etapas: primeiro $3^2 = 3+3+3 = 9$ (exibido no primeiro OUT); depois $3^3 = 9 \times 3 = 9+9+9 = 27$. As instruções 7 e 8 (SUB 3 e ADD 3) se cancelam, servindo apenas para ajustar o acumulador ao valor de dado disponível — uma limitação típica de não ter registrador extra. O valor final esperado é 27 (0x1B).

Sequência esperada do acumulador: $3 \rightarrow 6 \rightarrow 9$ (OUT = 9) $\rightarrow 9 \rightarrow 18 \rightarrow 27 \rightarrow 24 \rightarrow 27 \rightarrow 27$ (OUT = 27).

Ciclos esperados: 12 instruções $\times$ 6 estados = 72 ciclos.

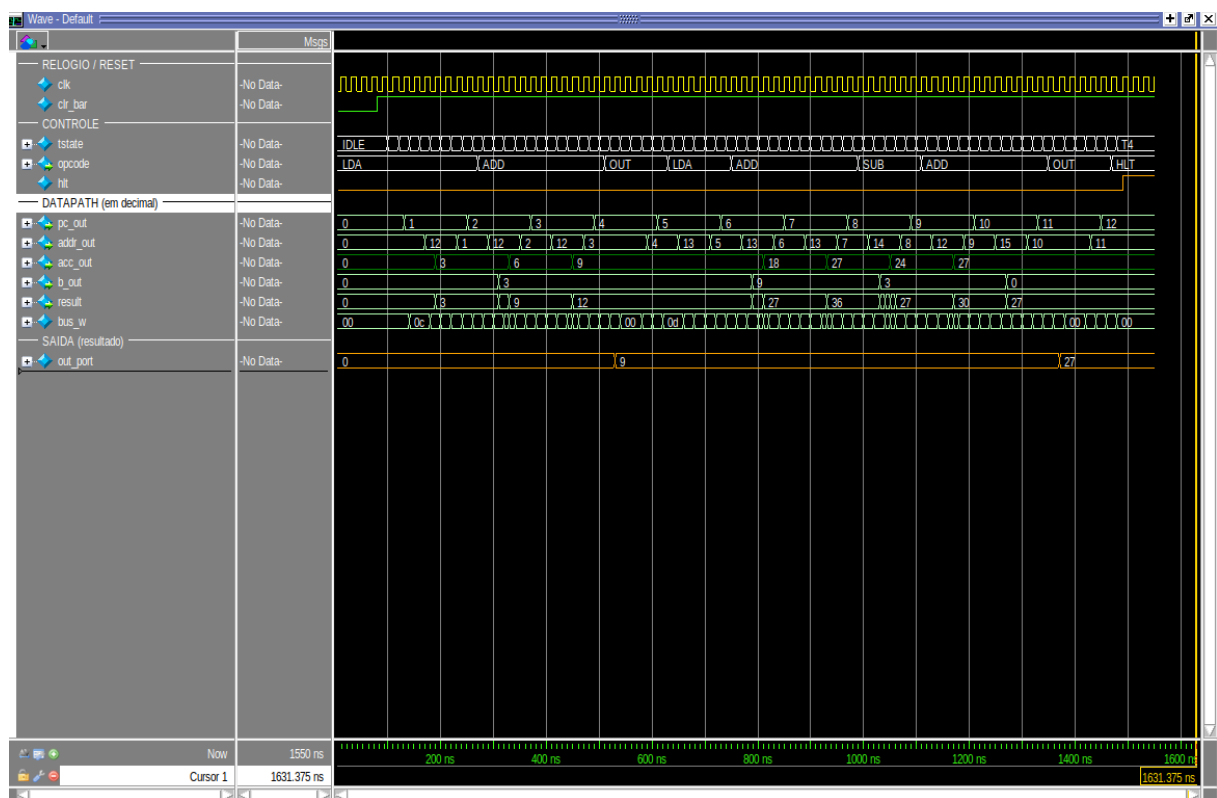


Figura 9 — 4.2. Programa 1 — $3^3 = 27$ — simulação completa.

O que a onda confirma: no acumulador (verde) vê-se exatamente 3, 6, 9 (primeiro OUT = 9) e, na segunda etapa, a subida até 27, com o segundo OUT exibindo 27. A saída (out_port) evolui $0 \rightarrow 9 \rightarrow 27$ e o processador para em HLT/T4. O assert do testbench confirma: obtido = esperado = 27.

4.3. Programa 2 — Expressão aritmética = 18

Cadeia de somas e subtrações: $((((7+3)-2)+5)-4, \text{ depois } +7-3+5; \text{ exibe } 9 \text{ e } 18$.

Listagem (endereço : instrução):

0 LDA 11 ; A <- 7	6 ADD 11 ; +7 -> 16
1 ADD 12 ; +3 -> 10	7 SUB 12 ; -3 -> 13
2 SUB 13 ; -2 -> 8	8 ADD 14 ; +5 -> 18
3 ADD 14 ; +5 -> 13	9 OUT ; mostra 18
4 SUB 15 ; -4 -> 9	10 HLT
5 OUT ; mostra 9	11..15 dados = 7, 3, 2, 5, 4

Resultado esperado (dedução): A expressão avaliada é $((7+3)-2)+5-4 = 9$ no primeiro trecho e, continuando a partir de 9, $((9+7)-3)+5 = 18$ no segundo. Cada operando é buscado por endereço na área de dados (posições 11 a 15). O valor final esperado é 18 (0x12).

Sequência esperada do acumulador: 7 → 10 → 8 → 13 → 9 (OUT = 9) → 16 → 13 → 18 (OUT = 18).

Ciclos esperados: 11 instruções × 6 estados = 66 ciclos (confirmado: HLT em 66 ciclos).

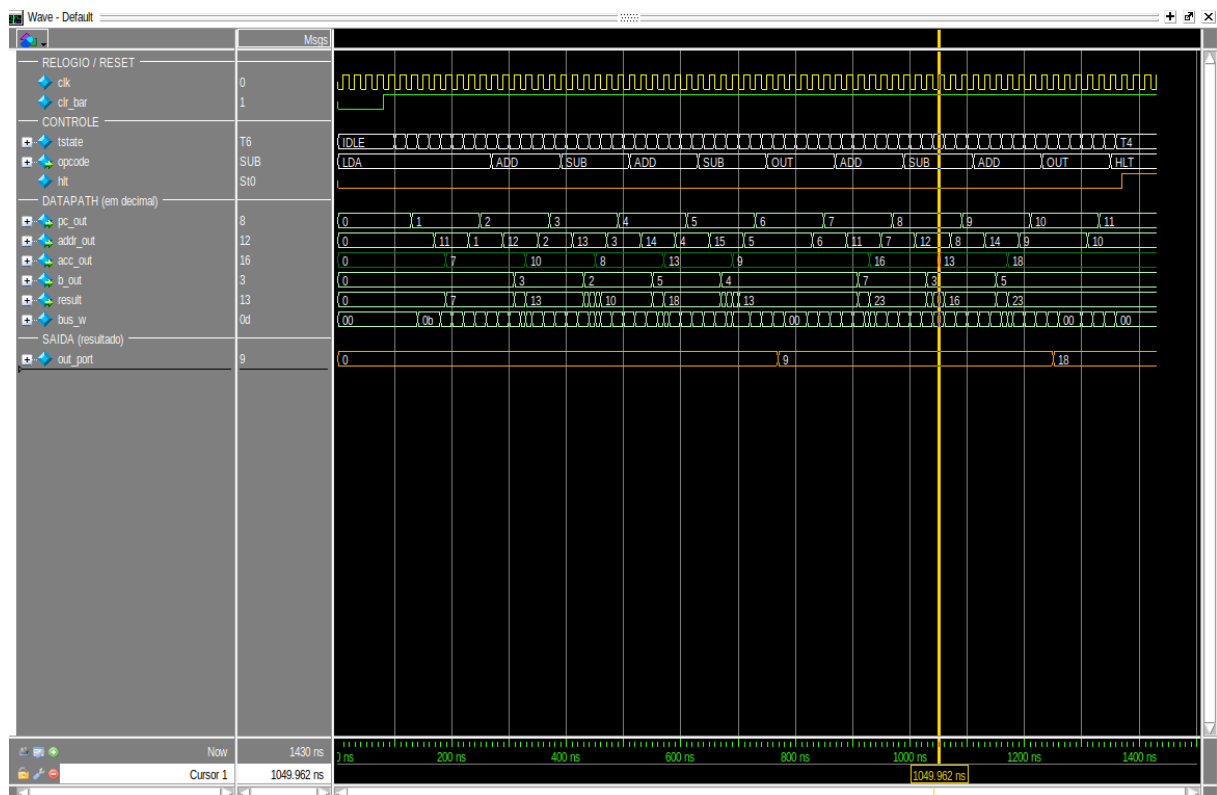


Figura 10 — 4.3. Programa 2 — Expressão aritmética = 18 — simulação completa.

O que a onda confirma: o acumulador percorre 7, 10, 8, 13, 9 e depois 16, 13, 18, batendo com a dedução termo a termo. Este é o programa que melhor mostra o encadeamento de operações e o uso de duas instruções OUT (resultados parciais). A saída evolui 0 → 9 → 18 e há parada em HLT/T4. Assert: obtido = esperado = 18.

4.4. Programa 3 — Multiplicação $3 \times 4 = 12$

Como não há instrução de multiplicação, faz-se por somas repetidas ($3+3+3+3$).

Listagem (endereço : instrução):

0	LDA 11 ; A <- 3	5	SUB 12 ; dado 0, A inalterado
1	ADD 11 ; A <- 6	6	ADD 13 ; dado 0, A inalterado
2	ADD 11 ; A <- 9	7	SUB 14 ; dado 0, A inalterado
3	ADD 11 ; A <- 12	8	ADD 15 ; dado 0, A inalterado
4	OUT ; mostra 12	9	OUT ; mostra 12
		10	HLT
		11	dado = 3

Resultado esperado (dedução): 3×4 significa somar o número 3 quatro vezes: $3+3+3+3 = 12$. Como o SAP-1 não tem laços (não há desvio), as quatro somas são escritas explicitamente. As instruções 5 a 8 operam com dados iguais a 0, de modo a não alterar o acumulador — servem apenas para preencher o programa até o segundo OUT. O valor final esperado é 12 ($0x0C$).

Sequência esperada do acumulador: $3 \rightarrow 6 \rightarrow 9 \rightarrow 12$ (OUT = 12) $\rightarrow 12 \rightarrow 12 \rightarrow 12$ (OUT = 12).

Ciclos esperados: 11 instruções $\times$ 6 estados = 66 ciclos (confirmado: HLT em 66 ciclos).

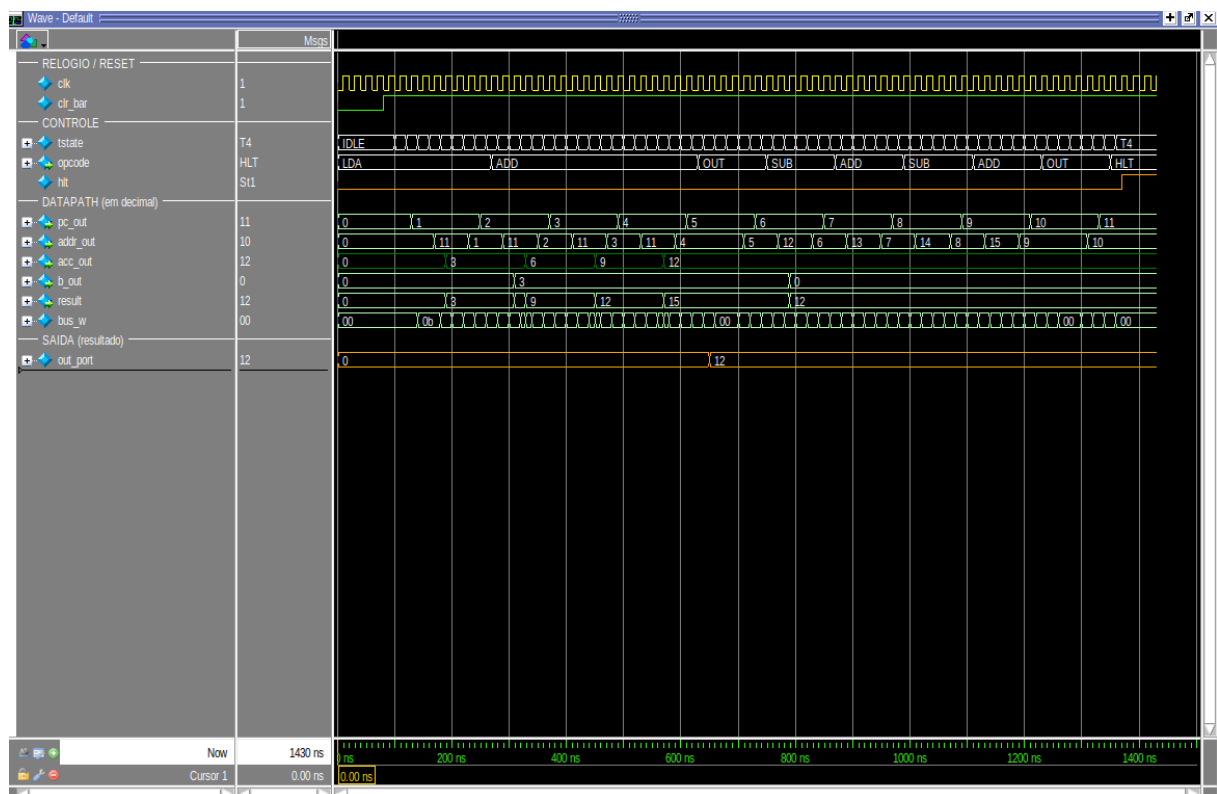


Figura 11 — 4.4. Programa 3 — Multiplicação $3 \times 4 = 12$ — simulação completa.

O que a onda confirma: a "assinatura" da multiplicação é inconfundível: o acumulador sobe de 3 em 3 (3, 6, 9, 12) e depois permanece constante (as somas/subtrações com 0). É a prova visual de que multiplicar, aqui, é repetir soma. A saída atinge 12 e há parada em HLT/T4. Assert: obtido = esperado = 12.

4.5. Programa 4 — Divisão $12 \div 4 = 3$

Divisão por subtrações repetidas: subtrai 4 até zerar e conta o quociente.

Listagem (endereço : instrução):

0	LDA 12 ; A <- 12	5	ADD 14 ; A <- 2
1	SUB 13 ; A <- 8	6	ADD 14 ; A <- 3 (quociente)
2	SUB 13 ; A <- 4	7	OUT ; mostra 3
3	SUB 13 ; A <- 0	8	HLT
4	LDA 14 ; A <- 1	12..14	dados = 12, 4, 1

Resultado esperado (dedução): $12 \div 4$ pergunta "quantas vezes o 4 cabe no 12?". A resposta é obtida subtraindo 4 repetidamente até zerar: $12 \rightarrow 8 \rightarrow 4 \rightarrow 0$, ou seja, 3 subtrações. Como não há como contar automaticamente (não há laço nem desvio), o quociente 3 é montado à mão, somando 1 três vezes. O número de subtrações é fixo para este caso específico — trocar os valores exigiria reescrever o programa. O valor final esperado é 3 (0x03).

Sequência esperada do acumulador: $12 \rightarrow 8 \rightarrow 4 \rightarrow 0$ (fim das subtrações) $\rightarrow 1 \rightarrow 2 \rightarrow 3$ (OUT = 3).

Ciclos esperados: 9 instruções $\times$ 6 estados = 54 ciclos (confirmado: HLT em 54 ciclos).

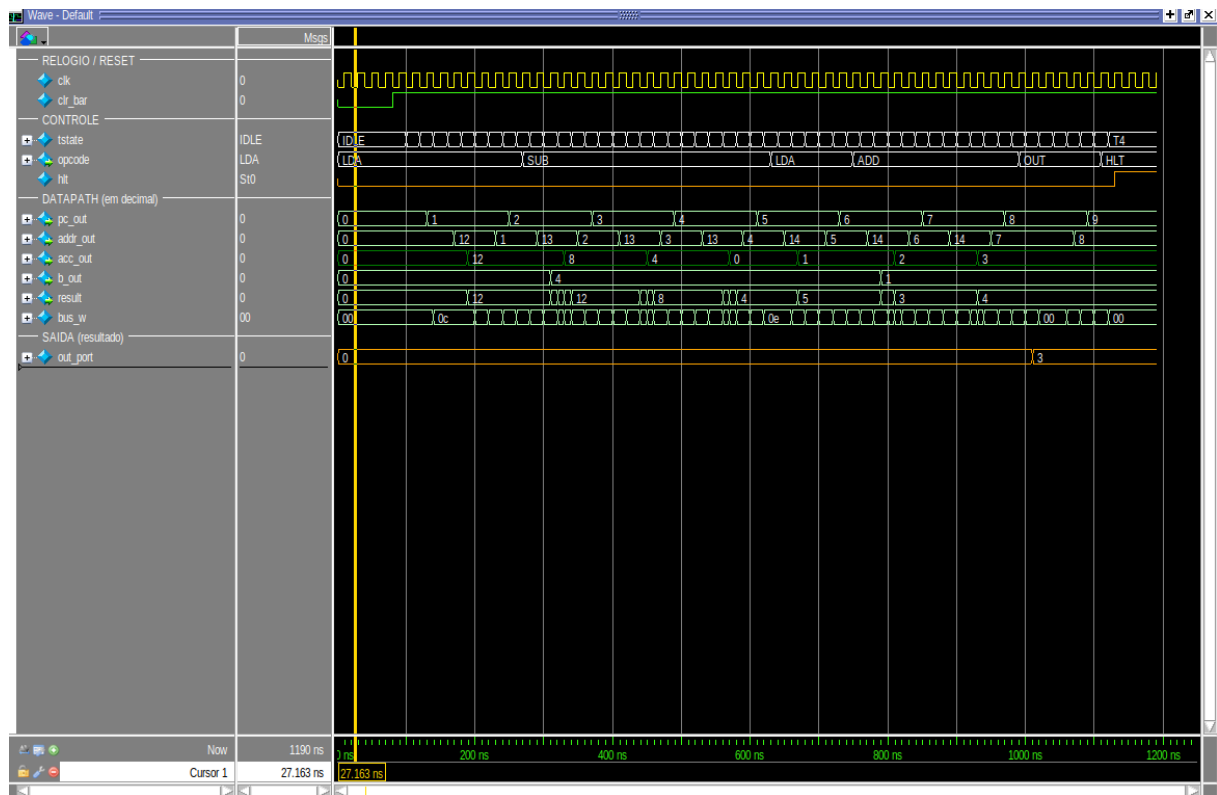


Figura 12 — 4.5. Programa 4 — Divisão $12 \div 4 = 3$ — simulação completa.

O que a onda confirma: a onda mostra o acumulador descendo de 4 em 4 (12, 8, 4, 0) — visualmente o oposto da multiplicação — e, em seguida, a contagem do quociente 1, 2, 3. A saída atinge 3 e há parada em HLT/T4. Assert: obtido = esperado = 3. Este caso evidencia bem a principal limitação do SAP-1: sem desvio condicional, o algoritmo não se adapta aos dados.

5. Resultados consolidados

Todos os 14 testbenches retornaram PASSOU, tanto no ModelSim ASE quanto no Icarus Verilog. Os quatro programas produziram exatamente a saída esperada e pararam corretamente em HLT. A tabela abaixo resume a verificação do sistema completo.

Prog.	Operação	Técnica	Esper.	Obt.	Ciclos	Parada
1	3^3	somas sucessivas	27	27	72	HLT/T4
2	expressão	soma/subtração encadeada	18	18	66	HLT/T4

		as				
3	3×4	somas repetidas	12	12	66	HLT/T4
4	$12 \div 4$	subtrações repetidas	3	3	54	HLT/T4

Resultado global: 14/14 testbenches PASSOU · 4/4 programas corretos.

6. Conclusão

A verificação por simulação confirmou o funcionamento correto do SAP-1 em dois níveis: os componentes, validados isoladamente por testbenches auto-verificáveis com formas de onda que evidenciam contagem, carga/retenção de registradores, leitura de memória e aritmética com wraparound; e o sistema completo, que executou os quatro programas produzindo os resultados esperados e parando corretamente em HLT. A combinação de simulação unitária e integrada, o uso de radix didáticos e a execução em lote deram confiança suficiente para prosseguir com a gravação na FPGA DE10-Lite. Como evolução natural, a inclusão de uma instrução de escrita (STA) e de desvios permitiria verificar laços e algoritmos com fluxo de controle, aproximando o projeto do SAP-2.